Proyecto MiBici: MLflow Project y tracking avanzado

Segunda entrega del proyecto integrador de MLOps

Tecnológico de Monterrey

Curso: TC5061.10 · Operaciones de aprendizaje automático

Equipo 09

Mariana Judith Carrera Rojas · A01840021

José Manuel Barrera Mendoza · A01797854

Luis Fernando Barranco Cruz · A01797868

David Carrera Castillo · A01840136

Fecha: 25 de octubre de 2026

Repositorio: https://github.com/a01840021-creator/TC5061-Proyecto-Equipo9

1. Resumen ejecutivo

Esta entrega transforma el proyecto MiBici en un flujo de experimentación reproducible mediante MLflow Projects, seguimiento centralizado de corridas y MLflow Model Registry (MLflow, s. f.-a, s. f.-b, s. f.-c). El proyecto responde tres preguntas operativas:

1. P1 (regresión): estimar los viajes que se iniciarán por estación y hora, con un horizonte de 24 horas.
2. P2 (clasificación): identificar estaciones en riesgo de desabasto de bicicletas o falta de anclajes durante las dos horas siguientes.
3. P3 (clustering): agrupar estaciones con patrones de utilización semejantes para apoyar la planeación urbana y operativa.

Todas las ejecuciones se lanzan desde el archivo MLproject. Los experimentos registran parámetros, métricas de validación y prueba, versión de datos, commit de Git, archivos de configuración, modelos con firma y ejemplos de entrada, así como visualizaciones específicas para cada tarea. Las búsquedas se organizan mediante *nested runs*, con una corrida padre por pregunta y una corrida hija por configuración.

La selección de modelos se realiza exclusivamente con métricas de validación para P1 y P2, reservando septiembre como prueba final. Para P3, al ser una tarea no supervisada, se utiliza el coeficiente Silhouette sobre los perfiles construidos antes del corte temporal. El mejor modelo de cada pregunta recibe el alias champion y la segunda mejor alternativa el alias challenger.

2. Datos, versionado y diseño temporal

El conjunto integra los viajes abiertos de MiBici de abril a septiembre de 2026 y la nomenclatura de estaciones correspondiente a septiembre de 2026. La ingesta descarga, normaliza y almacena los archivos en formato Parquet. El manifiesto references/manifiesto_datos.json conserva URLs, número de registros, hashes SHA-256 y una versión compacta de los datos.

La nomenclatura contiene 484 estaciones. Para P3 se evalúa adicionalmente una configuración que excluye estaciones sin salidas antes del corte; sus métricas se reportan por transparencia,

pero no compiten por los aliases debido a que se calculan sobre una población distinta.

2.1. Partición temporal

Conjunto	Periodo	Uso
Entrenamiento	1 de abril al 31 de julio de 2026	Ajuste de candidatos de P1 y P2
Validación	1 al 31 de agosto de 2026	Selección de hiperparámetros y modelos
Prueba	1 al 30 de septiembre de 2026	Evaluación final no utilizada para seleccionar
Clustering	Periodo anterior al 1 de septiembre	Construcción de perfiles de P3

La separación cronológica evita utilizar observaciones posteriores al momento de predicción. Las últimas horas de entrenamiento de P2 se purgan cuando su objetivo a dos horas cruzaría hacia validación. En P3, solamente se contabilizan llegadas que terminan antes del corte. Además, cada ventana se considera disponible 60 segundos después de su cierre, el mismo umbral del filtro de viajes espurios, para que ese filtro no use información posterior al momento de predicción. Estas decisiones corrigen fugas temporales que producirían estimaciones optimistas.

---

3. Proyecto reproducible con MLflow

El archivo python_env.yaml declara Python 3.10.20 y utiliza las versiones fijadas en requirements.txt. Se emplea uv como administrador del entorno porque permite a MLflow obtener la versión declarada de Python y crear el entorno sin depender de una instalación previa de pyenv. Los puntos de entrada se declaran en MLproject, de modo que cualquier integrante reproduce la secuencia completa con mlflow run (MLflow, s. f.-b).

3.1. Preparación del entorno

```
git clone https://github.com/a01840021-creator/TC5061-Proyecto-Equipo9.git
```

```
cd TC5061-Proyecto-Equipo9

uv venv .venv --python 3.10.20
source .venv/bin/activate
uv pip install -r requirements.txt
```

3.2. Servidor de tracking

```
mlflow server \
  --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./mlartifacts \
  --host 127.0.0.1 \
  --port 5000
```

En otra terminal:

```
cd TC5061-Proyecto-Equipo9
source .venv/bin/activate
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
```

3.3. Ejecución completa

```
mlflow run . --experiment-name mibici-equipo09 --env-manager uv -e ingest
--run-name ingest_datos
mlflow run . --experiment-name mibici-equipo09 --env-manager uv -e baseline
mlflow run . --experiment-name mibici-equipo09 --env-manager uv -e search -P
pregunta=p1
mlflow run . --experiment-name mibici-equipo09 --env-manager uv -e search -P
pregunta=p2
mlflow run . --experiment-name mibici-equipo09 --env-manager uv -e search -P
pregunta=p3
mlflow run . --experiment-name mibici-equipo09 --env-manager uv -e register
--run-name register_champion_challenger
```

El punto de entrada register no recibe Run ID manuales. Busca las corridas candidatas terminadas del commit actual, las ordena por la métrica de selección y registra automáticamente champion y challenger.

---

4. Estrategia de experimentación

Pregunta	Baseline oficial	Candidatos	Métrica de selección
P1	Persistencia mediante rezago semanal	Tres configuraciones de HistGradientBoosting	Menor val_mae

Pregunta	Baseline oficial	Candidatos	Métrica de selección
		y Ridge	
P2	Persistencia del saldo reciente	Tres configuraciones de HistGradientBoosting y regresión logística balanceada	Mayor val_average_precision
P3	K-Means con k=4	K-Means k=5 y k=6, MiniBatch K-Means k=5, Ward k=5 y K-Means k=5 sobre estaciones activas	Mayor train_silhouette, únicamente entre corridas con todas las estaciones

Para P1 se reportan MAE, RMSE y R^2. Para P2 se priorizan Average Precision, recall, precisión y F1 debido al desbalance de la clase positiva. Para P3 se consideran Silhouette, Davies-Bouldin, Calinski-Harabasz e inercia, complementadas con perfiles horarios y mapas. Los modelos y las métricas se implementan con scikit-learn (Pedregosa et al., 2011).

---

5. Resultados de P1: demanda por estación y hora

El baseline predice la demanda observada en la misma estación y hora de la semana anterior (rezago_168h). Los candidatos incluyen tres configuraciones de HistGradientBoostingRegressor y una regresión Ridge estandarizada.

Corrida	Modelo	Hiperparámetros clave	Val MAE	Test MAE	Test RMSE	Test R^2	Mejora de Test MAE frente al baseline
Baseline	Rezago semanal	rezago_168h	0.7812	0.9491	1.9700	0.4448	Referencia
Candidata 1	HistGradient Boosting	lr 0.05, 100 iter., 31 hojas, L2 0.0	0.7014	0.8461	1.6313	0.6193	+10.9 %

Corrida	Modelo	Hiperparámetros clave	Val MAE	Test MAE	Test RMSE	Test R^2	Mejora de Test MAE frente al baseline
Candidata 2	HistGradient Boosting	lr 0.10, 150 iter., 31 hojas, L2 0.1	0.6883	0.8328	1.6110	0.6287	+12.3 %
Candidata 3	HistGradient Boosting	lr 0.05, 200 iter., 63 hojas, L2 1.0	0.6869	0.8316	1.6061	0.6310	+12.4 %
Candidata 4	Ridge	alpha=1.0	0.7563	0.9066	1.7195	0.5770	+4.5 %

5.1. Análisis y selección

Champion: **Candidata 3, HistGradientBoosting (lr 0.05, 200 iteraciones, 63 hojas), val_mae 0.6869.**

Challenger: **Candidata 2, HistGradientBoosting (lr 0.10, 150 iteraciones, 31 hojas), val_mae 0.6883.**

La selección se hace con el MAE de validación porque expresa el error en viajes por estación y hora, la unidad con la que se decide cuántas bicicletas mover. Las tres configuraciones de HistGradientBoosting superan al rezago semanal en validación y en prueba, y conservan el mismo orden en ambas ventanas, lo que indica que la ventaja no proviene de haber afinado sobre agosto. El champion es la Candidata 3, con un MAE de validación de 0.6869 viajes, 12.1 % menor que el del baseline (0.7812). En septiembre reduce el MAE de prueba en 12.4 %, baja el RMSE de 1.9700 a 1.6061, lo que significa menos errores grandes para la planeación de flota, y eleva el R^2 de 0.4448 a 0.6310. Ridge mejora poco al baseline porque, con variables lineales, no captura la interacción entre hora, día y estación que los árboles sí aprenden. Todos los modelos, incluido el baseline, subestiman septiembre, que trae más viajes que los meses de entrenamiento; es un límite común y no de una configuración en particular. El challenger queda a 0.0014 viajes de error del champion y su corrida dura 8.8 s contra 11.8 s; con ambas por debajo de 15 s, el costo de cómputo no justifica hoy sacrificar precisión, y el challenger queda como respaldo si ese costo se vuelve relevante.

6. Resultados de P2: riesgo operativo

El baseline aplica una regla de persistencia basada en el saldo reciente respecto de la capacidad. Los candidatos incluyen tres configuraciones de HistGradientBoostingClassifier con pesos balanceados y una regresión logística estandarizada con class_weight="balanced".

Corrida	Modelo	Hiperparámetros clave	Val AP	Test AP	Test Recall	Test Precision	Test F1	Mejora de Test AP frente al baseline
Baseline	Persistencia del saldo	umbral de capacidad =0.30	0.0951	0.1013	0.0688	0.1872	0.1007	Referencia
Candidata 1	HistGradientBoosting	lr 0.05, 100 iter., 31 hojas, L2 0.0	0.2874	0.2859	0.8545	0.1533	0.2599	+182.2 %
Candidata 2	HistGradientBoosting	lr 0.10, 150 iter., 31 hojas, L2 0.1	0.3371	0.3267	0.8537	0.1639	0.2750	+222.5 %
Candidata 3	HistGradientBoosting	lr 0.05, 200 iter., 63 hojas, L2 1.0	0.3459	0.3331	0.8543	0.1658	0.2777	+228.8 %
Candidata 4	Regresión logística	C=1.0, balanceada	0.1460	0.1522	0.6886	0.1058	0.1834	+50.2 %

6.1. Análisis y selección

Champion: **Candidata 3, HistGradientBoosting (lr 0.05, 200 iteraciones, 63 hojas), val_average_precision 0.3459.**

Challenger: **Candidata 2, HistGradientBoosting (lr 0.10, 150 iteraciones, 31 hojas), val_average_precision 0.3371.**

La selección se basa en el Average Precision de validación porque resume el equilibrio entre precisión y recall sin quedar dominado por la clase mayoritaria, lo adecuado para una clase positiva de 4 a 5 % de prevalencia. El champion es la Candidata 3, con un Average Precision de validación de 0.3459, 263.7 % mayor que el del baseline (0.0951). En prueba conserva la

ventaja: su Average Precision (0.3331) y su F1 (0.2777) son los más altos, y su recall de 0.8543 está a dos diezmilésimas del mejor. Es decir, anticipa más de ocho de cada diez situaciones de riesgo, mientras que la regla de persistencia detecta menos de una de cada diez. Su precisión de 0.1658 implica que alrededor de una de cada seis alertas corresponde a un riesgo real; para la operación, ese costo en rutas es menor que el de no rebalancear una estación que se vacía o se llena, y el umbral de decisión puede ajustarse a la capacidad del equipo de rebalanceo. La diferencia entre validación y prueba es pequeña, señal de estabilidad entre meses. El challenger, con 150 iteraciones y 31 hojas, dura 15.1 s contra 20.0 s del champion, un ahorro marginal frente a la pérdida de Average Precision.

---

7. Resultados de P3: grupos de estaciones

El baseline utiliza K-Means con k=4. La búsqueda compara K-Means, MiniBatch K-Means y Ward. La variante de estaciones activas se analiza por separado porque utiliza una población distinta.

Corrida	Modelo	Configuración	Silhouette ↑	Davies-Bouldin ↓	Inercia ↓	Calinski-Harabasz ↑	Mejora de Silhouette frente al baseline
Baseline	K-Means	k=4, todas	0.1226	2.0381	9418.6	53.77	Referencia
Candidata 1	K-Means	k=5, todas	0.1287	1.6539	8926.5	49.07	+5.0 %
Candidata 2	K-Means	k=6, todas	0.1260	1.5236	8506.6	45.82	+2.8 %
Candidata 3	MiniBatch K-Means	k=5, todas	0.0833	2.2748	9460.0	40.21	-32.1 %
Candidata 4	Ward	k=5, todas	0.1192	1.6182	9257.2	43.04	-2.8 %
Análisis adicional	K-Means	k=5, activas	0.0962	1.7459	9502.2	32.47	No comparable directamente

7.1. Análisis y selección

Champion: **Candidata 1, K-Means con k=5 sobre las 484 estaciones, train_silhouette 0.1287.**

Challenger: **Candidata 2, K-Means con k=6 sobre las 484 estaciones, train_silhouette 0.1260.**

La comparación principal usa Silhouette entre corridas construidas con la misma población de 484 estaciones, porque valores mayores indican grupos más compactos y mejor separados. El champion es la Candidata 1, K-Means con cinco grupos, con un Silhouette de 0.1287, 5.0 % mayor que el del baseline con k=4 (0.1226). También mejora Davies-Bouldin (menor es mejor), de 2.0381 a 1.6539. Calinski-Harabasz (mayor es mejor) baja de 53.77 a 49.07 y la inercia de 9418.6 a 8926.5; ambas dependen del número de grupos, por lo que no deciden por sí solas. El challenger, K-Means con k=6, separa mejor según Davies-Bouldin (1.5236) a costa de fragmentar más las estaciones; MiniBatch K-Means pierde 32.1 % de Silhouette y Ward queda por debajo del baseline. Cinco grupos son manejables para definir protocolos de operación por tipo de estación, y K-Means es determinista con la semilla fija, lo que facilita reproducir la asignación. Un Silhouette cercano a 0.13 indica traslape entre grupos, por lo que los perfiles deben leerse junto con el mapa. La variante con estaciones activas (468 estaciones) se reporta como análisis complementario y no compite, porque modifica la población evaluada. Todas las corridas de P3 tardan entre 4 y 7 s, de modo que el costo no influye en la decisión.

Ward se adapta mediante WardCentroides: después del agrupamiento jerárquico calcula el centroide de cada grupo y asigna nuevas observaciones al centroide más cercano. Esto permite registrar el estimador con firma, cargarlo desde MLflow y utilizarlo en inferencia.

---

8. Model Registry y ciclo de vida

Se utilizan aliases porque las etapas clásicas Staging, Production y Archived están obsoletas desde MLflow 2.9; un alias es un puntero con nombre hacia una versión inmutable y se reasigna sin crear versiones nuevas (MLflow, s. f.-a). Para cada pregunta, register selecciona las corridas candidatas terminadas que pertenecen al commit actual:

Pregunta	Modelo registrado	Alias de producción	Alternativa
P1	mibici-equipo09-p1-demanda	champion (versión 1)	challenger (versión 2)
P2	mibici-equipo09-p2-riesgo	champion (versión 1)	challenger (versión 2)

Pregunta	Modelo registrado	Alias de producción	Alternativa
P3	mibici-equipo09-p3-grupos	champion (versión 1)	challenger (versión 2)

El alias champion representa el modelo aprobado para producción, mientras que challenger identifica la siguiente mejor versión para comparación posterior. Cada versión registra la pregunta, commit, versión de datos, corrida de origen, experimento, métrica de selección y valor de esa métrica (Figura 4). Versionar el modelo con su linaje hacia los datos y el código es la condición para reproducir y auditar un resultado (Lakshmanan et al., 2020; Treveil et al., 2020).

Los consumidores cargan el modelo sin fijar un número de versión:

```
import mlflow

mlflow.set_tracking_uri("http://127.0.0.1:5000")
modelo = mlflow.sklearn.load_model(
    "models:/mibici-equipo09-p1-demanda@champion"
)
```

La reasignación de aliases permite promover un nuevo modelo o regresar al anterior sin modificar la URI utilizada por el sistema consumidor.

9. Evidencias de MLflow

9.1. Comparación de corridas y nested runs

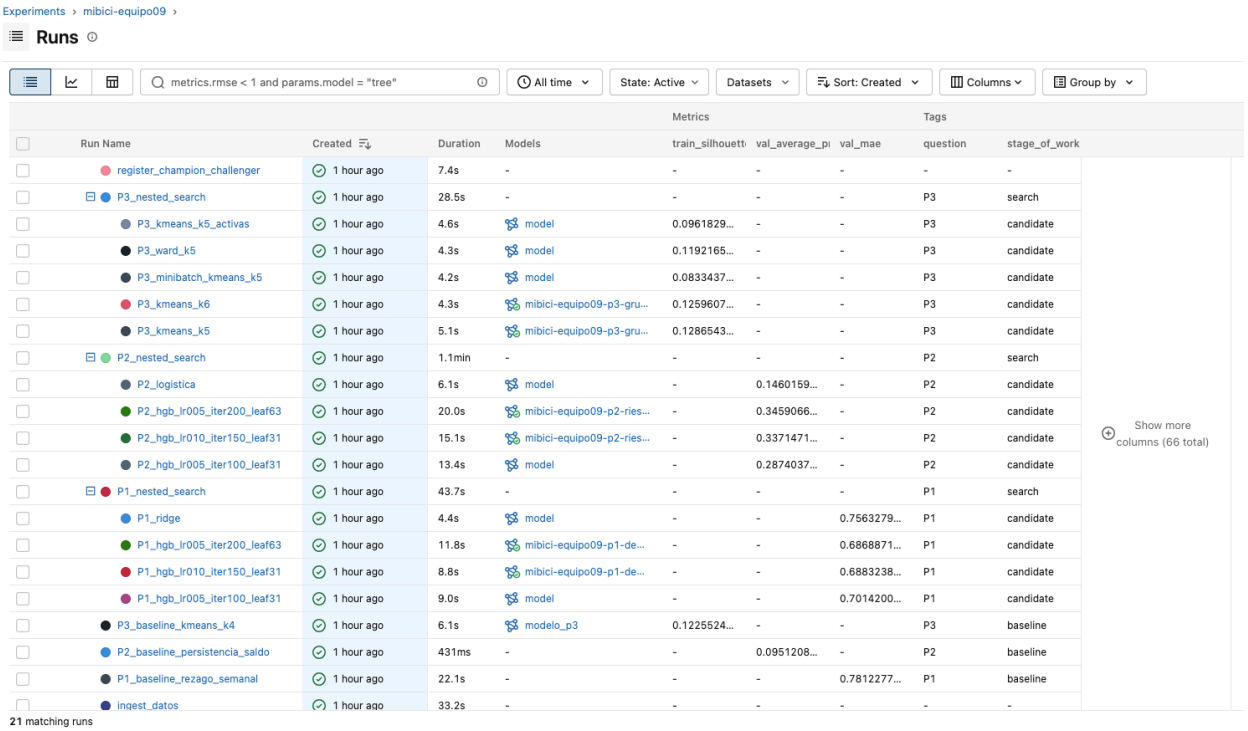


Figura 1. Experimento mibici-equipo09 en MLflow: corridas padre de búsqueda de P1, P2 y P3 con sus corridas hijas anidadas, junto con los baselines, la ingesta y el registro. Las columnas muestran la métrica de validación de cada pregunta y las etiquetas question y stage_of_work.

9.2. Artefactos de una corrida candidata

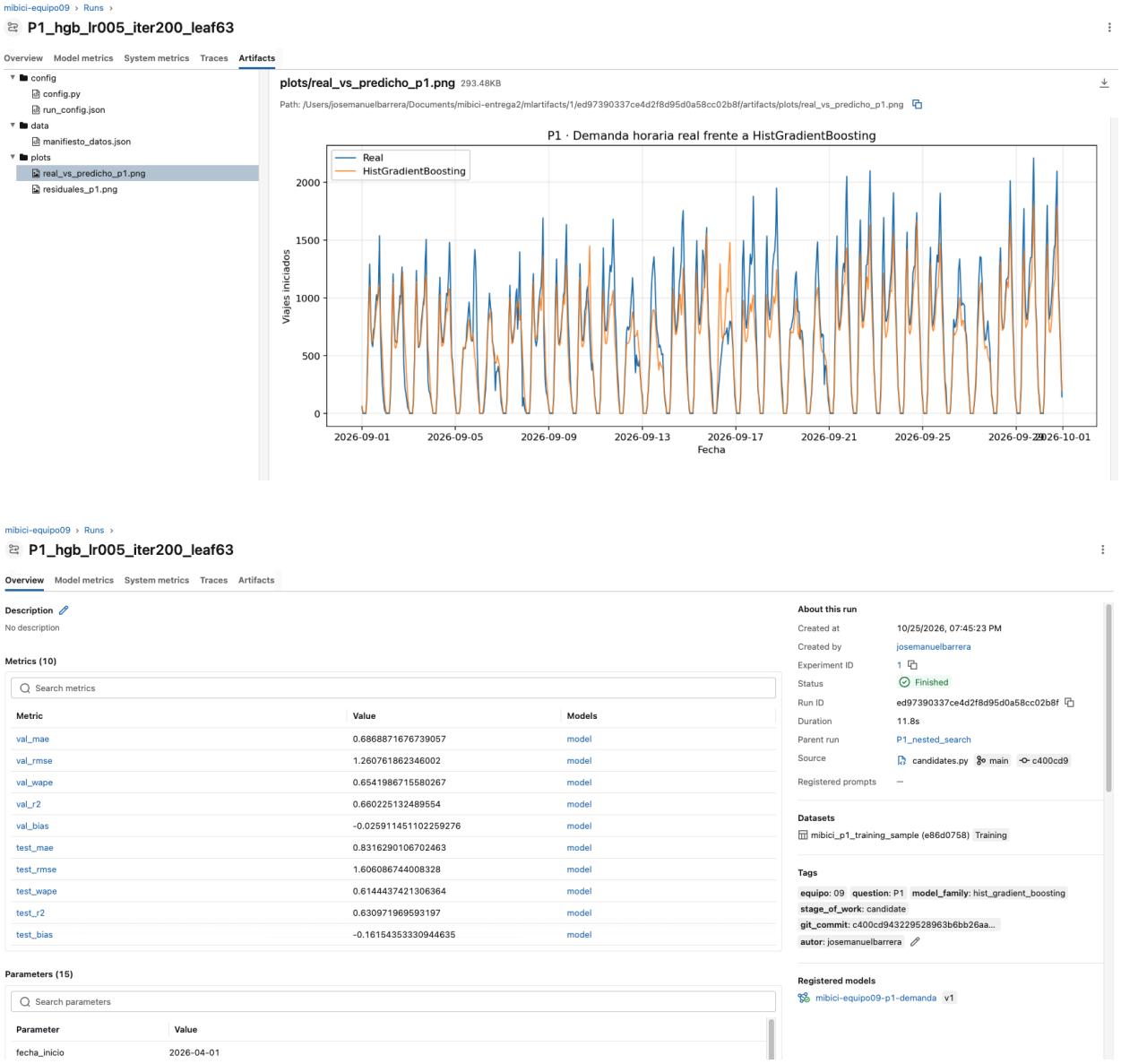


Figura 2. Corrida champion de P1. Arriba, sus artefactos organizados en config/, data/ y plots/, con la gráfica de demanda real contra predicha. Abajo, el detalle de la corrida: métricas de validación y prueba, parámetros, conjunto de datos registrado, etiquetas, commit de origen y modelo registrado.

9.3. Comparación visual de candidatos

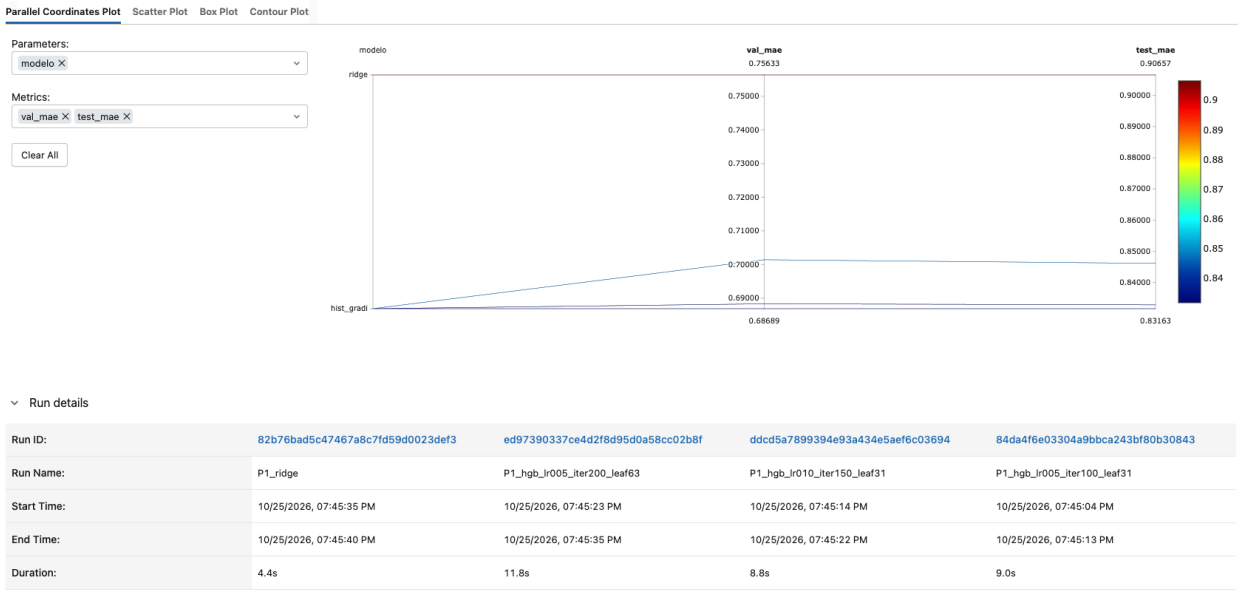


Figura 3. Comparación en MLflow de las cuatro candidatas de P1 con coordenadas paralelas de val_mae y test_mae. La selección usa val_mae; test_mae solo confirma el resultado en septiembre.

9.4. Model Registry y linaje

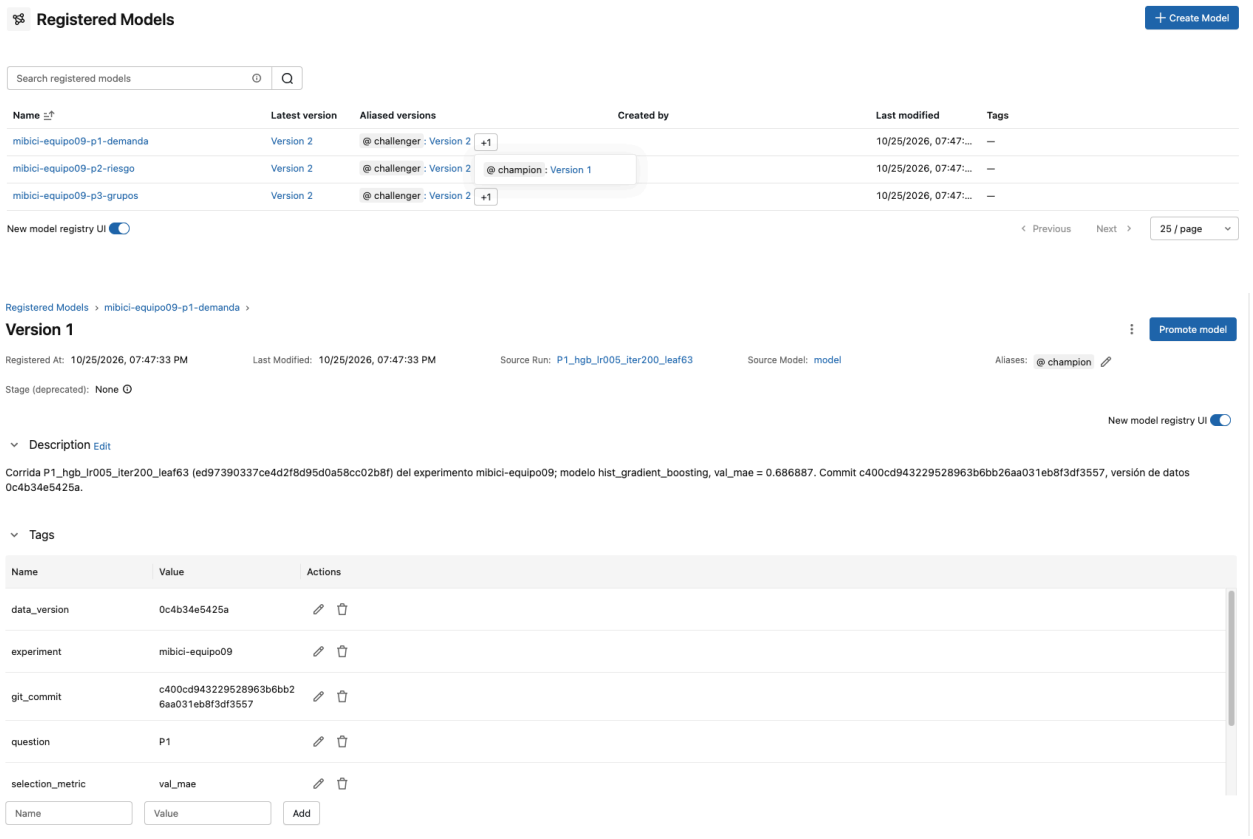


Figura 4. Arriba, los tres modelos registrados con su versión más reciente y sus aliases: challenger en la versión 2 y champion en la versión 1, que se muestra desplegado en P2. Abajo, la versión 1 de mibici-equipo09-p1-demanda con su alias champion, la corrida de origen, la descripción y las etiquetas de linaje (versión de datos, commit, pregunta y métrica de selección).

10. Limitaciones y trabajo futuro

- El periodo de seis meses permite validar la arquitectura, pero no representa un ciclo anual completo.
- P2 aproxima el riesgo mediante flujos y capacidad porque los datos abiertos no contienen inventario histórico por estación.
- P1 no incorpora todavía clima, días festivos, cierres viales o eventos masivos.
- Las métricas internas de P3 deben complementarse con validación geográfica y conocimiento operativo.

- La configuración local con SQLite es adecuada para la entrega; un escenario multiusuario requeriría una base compartida y almacenamiento remoto de artefactos.
- La variante de P3 con estaciones activas no puede compararse directamente con las corridas que utilizan las 484 estaciones.

Como trabajo futuro se propone ampliar el histórico, incorporar variables externas, calibrar el umbral operativo de P2 según costos y validar los grupos con personal de operación y planeación urbana.

11. Declaración de uso de inteligencia artificial

El equipo utilizó herramientas de inteligencia artificial generativa (ChatGPT y Codex de OpenAI, y Claude de Anthropic) como apoyo para revisar redacción, proponer estructuras documentales, depurar errores, interpretar mensajes de ejecución y sugerir fragmentos de código. El equipo revisó, adaptó, ejecutó y validó las contribuciones generadas antes de incorporarlas al repositorio y al reporte. La selección de datos, la definición de las preguntas, la ejecución de los experimentos, la interpretación de resultados y la responsabilidad sobre el contenido final corresponden al equipo.

12. Referencias

Lakshmanan, V., Robinson, S., & Munn, M. (2020). *Machine learning design patterns: Solutions to common challenges in data preparation, model building, and MLOps*. O'Reilly Media.

MLflow. (s. f.-a). *MLflow Model Registry*. https://mlflow.org/docs/latest/ml/model-registry/

MLflow. (s. f.-b). *MLflow Projects*. https://mlflow.org/docs/latest/ml/projects/

MLflow. (s. f.-c). *MLflow Tracking*. https://mlflow.org/docs/latest/ml/tracking/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Mivielle, M., & Roussat, N. (2020). *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media.